

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 670 — Information Retrieval

Satisfying an information need, which is not the same as matching a query

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 10 (with CSCE 638 and CSCE 676)

Prerequisites: CSCE 629 Analysis of Algorithms and CSCE 633 Machine Learning; CSCE 638 runs in parallel and supplies the representation material; CSCE 676 Module 02 for the hashing used in Module 08

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A search system over a collection you care about — with a hand-built relevance judgement set, a lexical baseline it has to beat, an evaluation that reflects how the results will be used, and a written account of the queries it fails

Course description

Retrieval begins with a person who wants something and has typed an approximation of it. **Module 01 establishes that gap properly**, because every technique in the course is an attempt to bridge it — **the query is evidence about a need rather than a specification of one**, and systems that treat it as a specification fail in predictable ways.

The course follows the engineering arc. **An inverted index, a scoring function derived from stated assumptions, efficient query processing, then learned ranking, then dense representations** — and **each step is a response to a measured limitation of the previous one**, which is more useful to understand than any single component.

The second theme is that **relevance is a human judgement and everything you measure is a proxy for it**. **Module 05 is the course's centre**: the metrics, where they come from, what they assume about user behaviour, and **why a system tuned on one proxy can be worse for the people using it** — which is CSCE 676 Module 13's proxy problem in this course's form.

The third is that **efficiency is a correctness concern**. **A ranking function you cannot evaluate within the latency budget is not a ranking function you have** — so Modules 04 and 08 are about making good ranking affordable, and the two-stage retrieve-then-rerank architecture that follows is the field's central practical idea.

And the closing position is about consequences and honest claims. **Module 12 covers what a ranking does to the things being ranked** — exposure, feedback, and the people behind the documents — and **Module 13 is about what a retrieval evaluation establishes**, because **'our NDCG improved' is a claim about a judgement set**.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Distinguish an information need from a query and from a document.
- 2 Build an inverted index and explain its compression.
- 3 Derive and apply the standard scoring functions.
- 4 Process queries efficiently with early termination.
- 5 Evaluate a retrieval system and defend the metric.
- 6 Train and deploy a learned ranker.
- 7 Explain dense retrieval and when it helps.
- 8 Search a large vector collection approximately.
- 9 Understand and reformulate queries.
- 10 Explain what web scale adds: crawling, spam, and links.
- 11 Build a retrieval-augmented generative system.
- 12 Explain exposure, feedback loops, and ranking's consequences.
- 13 State what a retrieval result honestly establishes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	The Information Need	What is the person actually trying to do?
2	The Index	How do you avoid reading every document?
3	Ranking	Why these formulas and not others?
4	Query Processing	How do you get the top ten without scoring everything?
5	Evaluation	Is your system better, or is your test set different?
6	Learning to Rank	How do you combine hundreds of relevance signals?
7	Dense Retrieval	Can you retrieve without sharing any terms?
8	Vector Search at Scale	How do you search a billion vectors in milliseconds?
9	Query Understanding	What can you do with the query before you search?
10	Web-Scale Retrieval	What does the web add that a document collection does not?

Wk	Module	Core question it answers
11	Generative Retrieval	What changes when the system answers instead of listing?
12	Exposure and Consequences	Who does your ranking affect, and how?
13	Claiming Relevance Honestly	Your NDCG improved. What follows?

Projects

Project 1 — An index and a ranker, built (after Module 6)

Build an inverted index and a BM25 ranker from scratch, and measure both correctness and speed.

Requirements

- **An inverted index you wrote**, with gap encoding and a compressed postings format (Module 02 §3).
- **BM25 implemented from the formula**, with the parameters' effect measured rather than defaulted (Module 03 §3).
- **Verified against a reference implementation** on the same collection — **the rankings should match**.
- **Query processing with early termination**, and the speedup measured against exhaustive scoring (Module 04 §3).
- **The index size reported** against the raw collection, with the compression attributed to each technique.
- **And a latency distribution**, at the median and the 95th percentile.

Done means

- **The rankings matching the reference** — which is binary, and is how you know the implementation is right.
- **The compression attributed per technique** rather than reported as a total, which is what demonstrates understanding.
- **The early termination verified safe: the top k must be identical to exhaustive scoring**, or the optimisation is a bug.
- **And the latency tail reported**, because **the median is not the number that determines whether a search system is usable**.

Project 2 — A search system, honestly evaluated (after Module 12)

Build a search system over a collection you care about, and evaluate it as if people depended on it.

Requirements

- **A collection, and at least thirty queries you wrote from real information needs** — not from the documents.
- **Relevance judgements you made yourself**, with your criteria written down before you judged (Module 05 §2).
- **A tuned BM25 baseline**, which the system has to beat (Module 05 §4).
- **Your system — learned ranking, dense retrieval, or hybrid — with the choice justified against its cost**.
- **An evaluation with the metric derived from how results will be used**, and the latency reported alongside.

- **And an analysis of the twenty worst queries**, with the failure categories drawn from them.

Done means

- **The queries written from needs rather than from documents** — **which is graded first, because a query derived from a document guarantees that document is retrievable** and the evaluation is then circular.
- **The judgement criteria written down in advance**, and applied consistently.
- **The baseline tuned**, with its parameters reported (CSCE 676 Module 13 §2).
- **And the twenty worst queries read individually**, with categories that came from the failures rather than from your expectations.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Manning, Raghavan & Schütze — Introduction to Information Retrieval (free)	The primary text, free in full from the authors. Modules 02 through 06 and 10 follow it closely, and its treatment of the index and the scoring derivations is still unmatched.
Croft, Metzler & Strohman — Search Engines: Information Retrieval in Practice (free PDF)	The systems companion, also free. Stronger on Modules 04, 09, and 10 — crawling, query processing, and what a production engine actually contains.
Lin, Nogueira & Yates — Pretrained Transformers for Text Ranking (free)	Modules 06, 07, and 11, free. The best available survey of the neural ranking material, and honest about which gains survived scrutiny.

Resource	Role
Zobel & Moffat — Inverted files for text search engines (free)	Module 02's reference. Everything about index construction and compression, from the people who measured it.
Sanderson — Test Collection Based Evaluation of IR Systems (free)	Module 05's history and its critique — how the evaluation methodology developed, and what it assumes that is not true.
The TREC proceedings, and BEIR (free)	The field's evaluation infrastructure. TREC for the methodology, BEIR for the zero-shot comparisons Module 07 relies on.

Tooling

- **A lexical search engine you can inspect: Lucene via PyLucene, or Anserini/Pyserini**, which is built for reproducible retrieval experiments and is what Module 05's exercises use.
- **And your own inverted index first. Module 02 requires you to build one**, because the compression and the skip structure are invisible from behind a library interface.
- **Python with sentence-transformers and faiss** for Modules 07 and 08 — and **read FAISS's index types rather than accepting the default**, which is Module 08 §4's exercise.
- **trec_eval, or ir_measures** for Module 05. **Use the standard implementation rather than writing your own metric**, because the edge cases are where the disagreements live.
- **A collection you care about and will judge yourself. Project 2 requires hand-made relevance judgements**, and making fifty of them teaches more about relevance than any reading.
- **And a latency budget you hold yourself to. Module 04's point is that a ranking you cannot compute in time does not exist** — so measure the latency from the first exercise onward.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.